

NAMA : Ilyas Najwan Pratama
NIM : J0403251081
KELAS : A2

Output Tugas ASD Praktikum 11

Screenshot hasil program

- Praktikum 1

```
=====
PRAKTIKUM 1 – Adjacency Matrix
=====

Adjacency Matrix Representation:
    0  1  2  3
-----
0 |  0  1  1  0
1 |  1  0  1  0
2 |  1  1  0  1
3 |  0  0  1  0

Node 0 terhubung ke node 1 dan node 2
Node 1 terhubung ke node 0 dan node 2
Node 2 terhubung ke node 0, node 1, dan node 3
Node 3 terhubung ke node 2
```

- Praktikum 2

```
=====
PRAKTIKUM 2 – Adjacency List (Undirected Graph)
=====

Adjacency List Representation:
A: B  C
B: A  D
C: A  D
D: B  C

A terhubung ke B dan C
B terhubung ke A dan D
C terhubung ke A dan D
D terhubung ke B dan C
```

- Praktikum 3

```
=====
PRAKTIKUM 3 – Konversi Matrix → Adjacency List
=====

Adjacency Matrix (input):
      0  1  2  3
0 |  0  1  1  0
1 |  1  0  1  0
2 |  1  1  0  1
3 |  0  0  1  0

Hasil Konversi – Adjacency List:
Node 0: 1  2
Node 1: 0  2
Node 2: 0  1  3
Node 3: 2
```

- Praktikum 4

```
=====
PRAKTIKUM 4 – Studi Kasus: Media Sosial
Digit akhir NIM: 1 → Media Sosial
=====

--- Adjacency List (Directed Graph Follow) ---

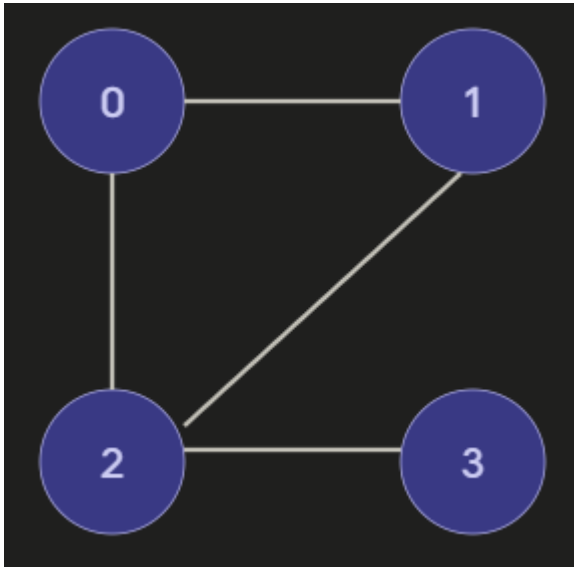
Siapa mem-follow siapa:
Andi   → Budi, Citra
Budi   → Dewi
Citra  → Dewi, Eko
Dewi   → Eko
Eko    → (tidak follow siapapun)

--- Adjacency Matrix (Directed Graph Follow) ---

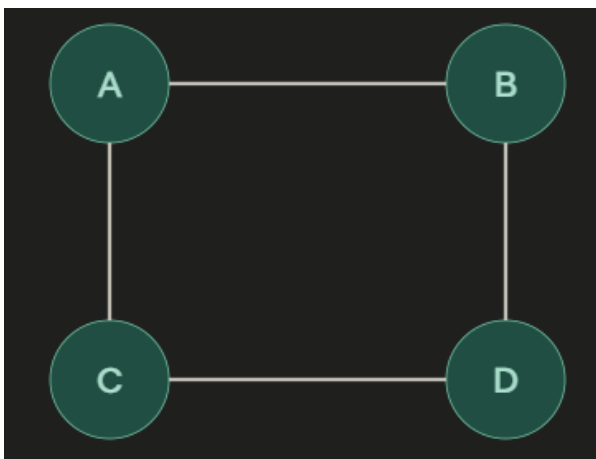
      Andi  Budi Citra  Dewi  Eko
-----
Andi |  0    1    1    0    0
Budi |  0    0    0    1    0
Citra|  0    0    0    1    1
Dewi |  0    0    0    0    1
Eko  |  0    0    0    0    0
```

Gambar/desain graph

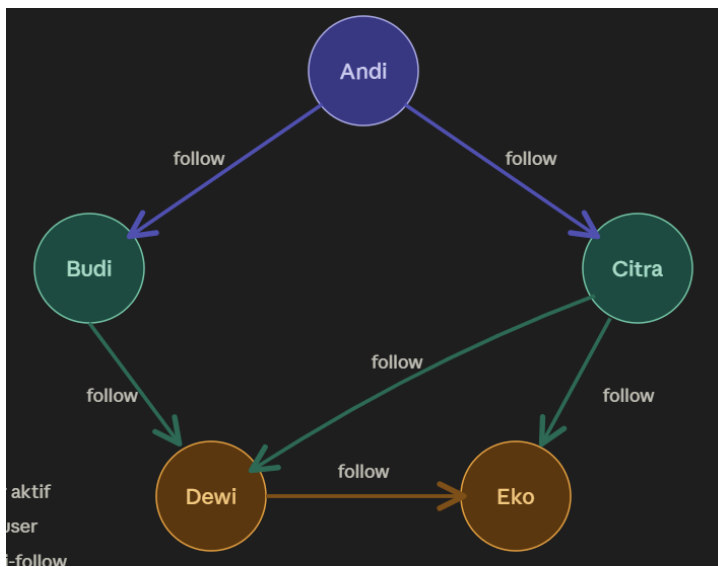
- Praktikum 1



- Praktikum 2



- Praktikum 4



Analisis Singkat Praktikum 4

Berdasarkan hasil eksekusi program representasi jaringan media sosial di atas, berikut adalah hasil analisis dari struktur graf yang terbentuk:

- 1. Jenis Graph: *Directed Graph* (Graf Berarah)
Studi kasus ini menggunakan *directed graph* karena interaksi di media sosial (sistem *follower/following*) umumnya bersifat satu arah. Pada kode program, jika Andi mem-follow Budi, hal tersebut tidak membuat Budi otomatis mem-follow balik Andi. Koneksi satu arah ini juga terbukti dari *output* Adjacency Matrix di mana letak angka 1 tidak simetris (baris Andi ke kolom Budi bernilai 1, tapi baris Budi ke kolom Andi bernilai 0).
- 2. Alasan Memilih Edge (Koneksi)
Saya menggunakan aksi "follow" sebagai *edge*. Alasannya, aktivitas *follow* adalah interaksi paling dasar dan utama yang membangun sebuah jaringan media sosial. Garis koneksi (*edge*) ini sangat penting untuk memetakan bagaimana arus informasi atau postingan menyebar dari satu pengguna ke pengguna lainnya.
- 3. Representasi yang Lebih Cocok: *Adjacency List*
Mengingat sifat jaringannya yang berupa *sparse graph*, penggunaan representasi Adjacency List jauh lebih efisien dan direkomendasikan. Jika kita memaksakan penggunaan *Adjacency Matrix*, sistem akan sangat boros memori karena harus memproses tabel berukuran besar yang isinya mayoritas hanya angka 0 (tidak ada relasi). *Adjacency List* lebih hemat ruang memori ($O(V+E)$) karena murni hanya mendaftarkan dan menyimpan koneksi yang benar-benar terbentuk saja, seperti yang ditampilkan pada hasil cetak program.
- 4. Kategori Graph: *Sparse Graph* (Graf Renggang)
Grafik yang terbentuk masuk ke dalam kategori *sparse graph* karena jumlah *edge* (6 koneksi) relatif sedikit dibandingkan total maksimal koneksi yang bisa terjadi antar 5 node. Dalam logika dunia nyata pun, dari jutaan akun yang ada di aplikasi, satu orang *user* biasanya hanya mem-follow segelintir akun saja. Karena mayoritas orang tidak saling terhubung, jaringannya akan selalu renggang.

Penjelasan Node dan Edge

Dalam struktur data, Graph pada dasarnya terdiri dari dua komponen utama, yaitu *node* dan *edge*.

- Node (atau Vertex): Ini adalah objek utama yang ada di dalam sebuah sistem. Misalnya, dalam kasus media sosial, yang menjadi *node* adalah pengguna atau *user*. Kalau di sistem peta, *node*-nya adalah nama-nama kota.

- **Edge:** Ini adalah garis atau hubungan yang menyambungkan antar *node* tersebut. Contohnya, di media sosial *edge* ini merepresentasikan status pertemanan atau aksi *follow*. Sedangkan di peta kota, *edge* ini mewakili jalan yang menghubungkan antar kota.

Analisis Adjacency List vs Matrix

Untuk merepresentasikan sebuah graph ke dalam bahasa kodingan seperti Python, ada dua cara yang paling umum dipakai, yaitu Adjacency Matrix dan Adjacency List. Dengan perbandingan sebagai berikut :

- **Adjacency Matrix:** Cara ini menggunakan tabel atau matriks 2 dimensi. Keunggulan utamanya adalah proses pengecekan koneksi (*edge*) antar *node* yang sangat cepat dengan kompleksitas waktu $O(1)$. Namun kekurangannya, cara ini sangat boros penggunaan memori karena membutuhkan ruang memori sebesar $O(V^2)$. Oleh karena itu, matriks ini cuma cocok dipakai buat *dense graph* atau grafik yang koneksinya sangat padat.
- **Adjacency List:** Cara ini menyimpan data dalam bentuk daftar (*list*) yang isinya adalah tetangga dari masing-masing *node*. Kelebihan terbesarnya adalah jauh lebih hemat memori komputer dengan kompleksitas ruang $O(V+E)$, karena program hanya mencatat koneksi yang benar-benar ada. Kekurangannya, kalau kita mau mengecek satu *edge* secara spesifik, prosesnya sedikit lebih lambat dibanding matriks. Adjacency List ini sangat cocok dipakai untuk *sparse graph* (grafik renggang).

Kesimpulan Singkat

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data graph sangat berguna untuk memodelkan hubungan antar objek pada kasus dunia nyata. Kita bisa menerjemahkan kasus tersebut ke dalam program Python menggunakan Adjacency Matrix ataupun Adjacency List. Pemahaman tentang cara menyimpan graph dan membaca *adjacency list* ini menjadi pondasi yang sangat penting untuk mempelajari algoritma *traversal* pada praktikum selanjutnya, yaitu pencarian rute menggunakan algoritma BFS (Breadth First Search) dan DFS (Depth First Search).